

Etapa 1 - Analizador Léxico de MiniJava

Compiladores e Intérpretes — UNS, 2026

Implementación de un analizador lexico para MiniJava junto con un módulo principal de consola. El analizador se construye a partir de un autómata finito determinista implementado manualmente (un método por estado).

1 Estructura del proyecto

```
codigo/
|-- Compilar.sh / Compilar.bat           Compila y genera Compilador.jar
|-- CorrerTests.sh / CorrerTests.bat     Compila y ejecuta los testers JUnit
|-- lib/                                 Jars de JUnit 4 y Hamcrest (se autodescargan)
|-- src/main/java/
|   |-- moduloprincipal/
|   |   \-- ModuloPrincipal.java         Modulo principal (interfaz con el usuario)
|   |-- analizadorlexico/
|   |   |-- AnalizadorLexico.java        Automata finito determinista
|   |   |-- Token.java                  Representacion de un token (nombre, lexema, linea)
|   |   \-- ExcepcionLexica.java         Excepcion propagada ante un error lexico
|   \-- sourcemanager/
|       |-- SourceManager.java           Interfaz provista por la catedra
|       |-- SourceManagerImpl.java       Implementacion provista (por lineas, referencia)
|       \-- SourceManagerEficiente.java  Implementacion propia por caracteres (se usa)
|-- src/test/java/test/                 Testers JUnit provistos por la catedra
\-- resources/
    |-- sinErrores/                     Casos de prueba sin errores
    \-- conErrores/                     Casos de prueba con errores
```

2 Componentes

2.1 Módulo principal (ModuloPrincipal)

Recibe por parámetro la ruta del archivo fuente MiniJava (acepta cualquier extensión):

```
java -jar Compilador.jar programa1.java
```

Asocia el archivo a un `SourceManagerEficiente`, crea el analizador léxico y le solicita tokens hasta recibir el token de fin de archivo. Cada token se imprime por pantalla con el formato `(NombreToken, Lexema, NroLinea)`. Al finalizar sin errores se imprime `[SinErrores]`.

Ante un error léxico, la excepción `ExcepcionLexica` es capturada por el módulo principal, que reporta el mensaje de error y el código de error, y continúa solicitando tokens hasta el EOF sin finalizar la ejecución (multi-detección de errores, ver sección Logros). Sólo si la corrida completa no registró errores se imprime `[SinErrores]`.

2.2 Manejador de archivos eficiente (SourceManagerEficiente)

Implementa la interfaz `SourceManager` leyendo el fuente de a caracteres (en lugar de líneas como la implementación provista). Características:

- Lectura por caracteres con `InputStreamReader` (UTF-8) sobre `PushbackReader`.

- Reconoce las tres convenciones de salto de línea (LF, CR y CRLF), normalizándolas a un único `\n`. La secuencia `\r\n` no se contabiliza como dos saltos.
- `getLineNumber()` devuelve la línea del último carácter devuelto. Un carácter de salto de línea pertenece a la línea que termina, y el número de línea se incrementa al leer el siguiente carácter. De esta forma el token EOF reporta la línea del final físico del archivo (por ejemplo, un archivo de 21 líneas que termina con salto de línea reporta el EOF en la línea 22).

La implementación provista por la cátedra (`SourceManagerImpl`) se incluye como referencia, pero no se utiliza en la ejecución porque su conteo de líneas difiere en el token EOF cuando el archivo termina con un salto de línea.

2.3 Analizador lexico (AnalizadorLexico)

Autómata finito determinista implementado con un método por estado. El estado inicial `e0` descarta espacios en blanco y comentarios, y despacha al estado correspondiente según el primer carácter del lexema. Cada método consulta `caracterActual` (un carácter de adelanto que nunca se consume al finalizar un token), lo agrega al lexema y avanza leyendo el próximo carácter.

Las palabras reservadas se resuelven consultando una tabla: al construir un identificador de método o variable se verifica si su lexema corresponde a una palabra reservada y, en ese caso, se devuelve el token de palabra reservada correspondiente.

3 Tokens del lenguaje

Se adoptaron los siguientes nombres para los tokens.

3.1 Palabras reservadas

Se reconocen con un identificador de método o variable y se devuelven con nombre `pr_<palabra>`:

```
class extends interface implements static boolean char int void public if else while
return var this new null true false
```

Los nombres de tokens siguen el ejemplo de las pautas de la Etapa 1: `litString` para los literales string, `idMV` para identificadores de método o variable, `pr_<palabra>` para palabras reservadas, `op<simbolo>` para operadores y `llaveC` para puntuación.

3.2 Identificadores

Token	Expresión regular	Descripción
<code>idClase</code>	<code>[A-Z] [A-Za-z0-9_]+</code>	Clase o interfaz
<code>idGen</code>	<code>[A-Z]</code>	Parámetro de tipo genérico (una sola mayúscula)
<code>idMV</code>	<code>[a-z] [A-Za-z0-9_]*</code>	Método o variable

3.3 Literales

Token	Expresión regular	Restricciones
<code>litInt</code>	<code>[0-9]{1,9}</code>	Máximo 9 dígitos; más de 9 es error
<code>litChar</code>	<code>'[^\\'\n]'</code> o <code>'\\'[^\\n]'</code>	Un carácter normal, o barra invertida seguida de cualquier carácter
<code>litString</code>	<code>"([^\\"\\n] \\'[^\\n])"</code>	Sin saltos de línea; <code>\n</code> no cierra el literal

Los literales `true`, `false` y `null` se representan con las palabras reservadas `pr_true`, `pr_false` y `pr_null`.

3.4 Operadores

op+ op- op* op/ op% op< op> op! op= op== op>= op<= op!= op&& op|| op++ op--

Los operadores se reconocen con el criterio del lexema más largo (por ejemplo, >= antes que >). Los caracteres & y | no forman operadores por sí solos: un & o | no seguido de otro igual es un error léxico.

3.5 Puntuación

parA (	parC)	llaveA {	llaveC }	corcheteA [	corcheteC]
puntoComa ;	coma ,	punto .	dosPuntos :		

3.6 Fin de archivo

Token EOF con lexema \$.

4 Elementos ignorados

- Espacios en blanco: espacio, tabulador y salto de línea (LF, CR o CRLF).
- Comentarios de una línea: desde // hasta el final de la línea.
- Comentarios multilínea: desde /* hasta */.

5 Errores lexicos

Al detectar un error se lanza `ExcepcionLexica` con el lexema armado hasta el momento y el número de línea. El módulo principal la captura, reporta el error y continúa pidiendo tokens hasta el EOF, de forma que una corrida reporta todos los errores léxicos del fuente.

Error Lexico en linea <N>: <mensaje>
[Error:<lexema>|<N>]

Para que el código de error quede en una sola línea, en el caso de un comentario multilínea sin cerrar el lexema reportado es el que abre el comentario, /*.

Mensajes según la naturaleza del error:

Caso	Mensaje
Caracter inválido (no es símbolo válido)	<lexema> no es un simbolo valido
String sin comilla de cierre	literal string mal formado
Caracter mal formado	literal caracter mal formado
Entero con más de 9 dígitos	literal entero demasiado largo
Comentario multilínea sin cerrar	comentario sin cerrar

6 Formato de salida

6.1 Análisis exitoso

Un token por línea con el formato (NombreToken, Lexema, NroLinea). El lexema incluye los delimitadores para strings y caracteres (por ejemplo, (litString, "hola", 2) o (litChar, '\n', 3)). Al final se imprime el token de fin de archivo y:

[SinErrores]

6.2 Análisis con errores

Cada error se reporta en el momento en que se detecta, con el formato:

```
Error Lexico en linea 2: # no es un simbolo valido
[Error:#|2]
```

Tras cada error el análisis continúa, por lo que pueden reportarse varios errores en una misma corrida (por ejemplo, `lexConErrores02.java` reporta el string sin cerrar de la línea 3 y luego la comilla sin cerrar de la línea 4). Si se detectó al menos un error no se imprime `[SinErrores]`.

Toda la salida se realiza con `System.out`.

7 Compilacion y ejecucion

No se usa ninguna herramienta de build: solo se necesitan `javac`, `jar` y `java` de cualquier JDK 21 o superior (probado con JDK 25).

```
./Compilar.sh          # compila y genera Compilador.jar (Windows: Compilar.bat)
java -jar Compilador.jar resources/sinErrores/lexSinErrores01.java
./CorrerTests.sh       # compila y ejecuta los testers JUnit (Windows: CorrerTests.bat)
```

`CorrerTests.sh` descarga automáticamente `junit-4.13.2.jar` y `hamcrest-core-1.3.jar` en `lib/` si no están (requiere `curl`); si no hay red, se pueden descargar a mano de Maven Central y dejarlos en esa carpeta.

7.1 Comandos manuales equivalentes

Compilar (desde `codigo/`):

```
javac -d build/classes \
  src/main/java/moduloprincipal/*.java \
  src/main/java/analizadorlexico/*.java \
  src/main/java/sourcemanager/*.java
jar cfe Compilador.jar moduloprincipal.ModuloPrincipal -C build/classes .
```

Correr los tests (desde `codigo/`, con JUnit 4.13.2 y Hamcrest 1.3 en `lib/`):

```
javac -cp "build/classes:lib/junit-4.13.2.jar:lib/hamcrest-core-1.3.jar" \
  -d build/test-classes src/test/java/test/*.java

java -cp "build/classes:build/test-classes:lib/junit-4.13.2.jar:\
lib/hamcrest-core-1.3.jar" org.junit.runner.JUnitCore \
  test.TesterDeCasosSinErrores test.TesterDeCasosConErrores
```

En Windows el separador del classpath es `;` en lugar de `:`.

Los testers (`TesterDeCasosSinErrores` y `TesterDeCasosConErrores`) usan el working directory `codigo/` para acceder a `resources/sinErrores/` y `resources/conErrores/` (los scripts ya se posicionan ahí; los comandos manuales deben correrse desde `codigo/`).

8 Tests provistos por la catedra

Casos de prueba incluidos (copiados del material de la cátedra):

- `resources/sinErrores/lexSinErrores01.java` - identificadores, strings, chars y EOF.
- `resources/sinErrores/lexSinErrores02.java` - ejemplo de la convención de salida.

- `resources/sinErrores/lexSinErrores03.java` - archivo solo con comentarios.
- `resources/conErrores/lexConErrores01.java` - caracter inválido #.
- `resources/conErrores/lexConErrores02.java` - string sin comilla de cierre.
- `resources/conErrores/lexConErrores03.java` - múltiples errores en una corrida.

9 Logros

- **Manejador de Archivos Eficiente:** `SourceManagerEficiente` lee de a caracteres y conforma con la interfaz `SourceManager`, sin modificar la interfaz provista.
- **Multi-detección de Errores Léxicos:** el módulo principal no finaliza ante el primer error léxico; captura la excepción, reporta el error y continúa pidiendo tokens hasta el EOF. El léxico consume el carácter ofensor en los estados de error para garantizar el progreso de la recuperación. Sólo si la corrida no registró errores se imprime `[SinErrores]`.